SAMSUNG INNOVATION CAMPUS

BIG DATA COURSE · NHÓM 6

BÁO CÁO KỸ THUẬT CHI TIẾT

SentimentGuard

Phân loại cảm xúc bình luận trực tuyến theo thời gian thực để phát hiện sớm các đợt bùng phát bình luận độc hại

Giải thích chương trình, mô hình và cách hệ thống hoạt động

Nhóm thực hiện	Nhóm 6
Thành viên	Vàng A Ký (Nhóm trưởng); Nguyễn Đình Bằng; Vàng Thị Dẳm; Dương Thị Hạnh; Ngô Lương Nguyên
Đề tài	Mô hình phân loại cảm xúc dựa trên bình luận trực tuyến
Phiên bản	1.0
Ngày hoàn thiện	19/10/2026
Phạm vi	Tài liệu kỹ thuật nội bộ, viết bằng tiếng Việt

> **Lưu ý về phạm vi tài liệu**
> Đây là tài liệu giải thích kỹ thuật bằng tiếng Việt. Bản nộp chính thức cho giảng viên là `deliverables/Final_Report.docx` (tiếng Anh), điền theo đúng template của cô. Mọi số liệu trong tài liệu này được lấy từ các file kết quả thật trong `outputs/`; không có số liệu nào được đặt tay. Phần dữ liệu streaming là **dữ liệu phát lại (replay)** từ review IMDb công khai, **không phải bình luận livestream thật**.

Mục lục

1. Tóm tắt điều hành....3
2. Bài toán, mục tiêu và phạm vi....4
3. Kiến trúc hệ thống và luồng dữ liệu....6
4. Dữ liệu: nguồn, làm sạch và chia tập....8
5. Mô hình và quy trình huấn luyện....11
6. Kết quả đánh giá mô hình....14
7. Phân tích lỗi và diễn giải....17
8. Hệ thống streaming thời gian thực....20
9. Kiểm chứng streaming....23
10. Phân tích khám phá dữ liệu và dashboard....26
11. Chương trình thử nghiệm mô hình....28
12. Hướng dẫn chạy lại....29
13. Hạn chế, rủi ro và hướng phát triển....31
14. Phụ lục....32

Cách đọc tài liệu

Các chương 1–3 trả lời câu hỏi “làm gì và vì sao”. Chương 4–7 đi sâu vào dữ liệu, mô hình và kết quả. Chương 8–10 giải thích luồng streaming, cách kiểm chứng và dashboard. Chương 11–12 dành cho người muốn chạy lại chương trình. Chương 13 ghi rõ giới hạn — nên đọc trước khi báo cáo. Số trang trong mục lục là số trang của bản PDF đi kèm; bản Word có thể lệch 1 trang tùy font và cách ngắt trang của Word.

1. Tóm tắt điều hành

SentimentGuard là một nguyên mẫu hệ thống phân loại cảm xúc bình luận trực tuyến theo thời gian thực, nhằm **phát hiện sớm các đợt bùng phát bình luận tiêu cực** trong một phiên livestream hoặc một cộng đồng trực tuyến. Hệ thống nhận từng bình luận, dự đoán một trong ba nhãn **negative / neutral / positive**, ghi kết quả ra Kafka và Parquet, tổng hợp theo cửa sổ thời gian, rồi hiển thị trên dashboard kèm quy tắc cảnh báo.

Điểm cốt lõi của đồ án không phải là một mô hình "cao điểm", mà là **một đường ống hoàn chỉnh và trung thực**: dữ liệu được làm sạch và loại trùng trước khi chia tập; TF-IDF chỉ được học trên tập train và nằm trong Spark ML Pipeline; tập test chỉ dùng một lần; streaming dùng event time, watermark và hàng đợi lỗi (DLQ); mọi con số trong báo cáo đều truy được về file kết quả thật.

Kết quả chính: mô hình **LogisticRegression** đạt **accuracy 70,18%**, **macro-F1 62,33%** và **weighted-F1 72,13%** trên 27.792 bình luận test. Trong đó F1 lớp positive là **0,819**, negative **0,672**, neutral **0,378** — neutral là lớp yếu nhất vì nhãn được suy từ rating 5–6 của IMDb nên rất nhiễu. Trên luồng streaming, lần chạy `verify1` đã gửi **1.024** bản ghi: **1.014** bản ghi được chấm điểm và **10** bản ghi lỗi vào DLQ; **1.000/1.000** dự đoán trên luồng trùng khớp với dự đoán offline trên cùng `comment_id`.

Hệ thống **chưa sẵn sàng cho môi trường sản xuất**. Dữ liệu streaming là replay từ review IMDb công khai, không phải bình luận livestream thật; mô hình có thể đang đọc điểm số dạng "6/10" thay vì hiểu cảm xúc; và cơ chế exactly-once **chưa được kiểm chứng** — chỉ dám khẳng định at-least-once cho phần đã đo. Những giới hạn này được trình bày ở chương 13.

Bảng 1. Các con số then chốt của đồ án (nguồn: outputs/tables/)

Chỉ số	Giá trị	Nguồn
Bình luận sạch sau làm sạch + loại trùng	185.286	cleaning_log.csv
Train / validation / test	129.701 / 27.793 / 27.792	class_distribution.csv
Số đặc trưng TF-IDF	20.000	model_results.csv
Mô hình được chọn	LogisticRegression	model_results.csv
Accuracy trên test	70,18%	model_results.csv
Macro-F1 trên test	62,33%	model_results.csv
F1 negative / neutral / positive	0,672 / 0,378 / 0,819	per_class_metrics.csv
Bản ghi streaming đã chấm / vào DLQ	1.014 / 10	stream_verification.csv
Dự đoán luồng khớp offline	1.000/1.000 (100%)	stream_verification.csv
Độ trễ p50 / p95 / p99	2.413 / 9.663 / 27.091 ms	stream_verification.csv
Số streaming query	6	stream_batches.csv
Số test tự động	77 passed	pytest (15/10/2026)

> **Cam kết trung thực**
> Tài liệu này không dùng accuracy làm chỉ tiêu duy nhất, không gọi dữ liệu replay là dữ liệu livestream thật, và không khẳng định exactly-once. Chỗ nào chưa đo được, tài liệu ghi rõ **"chưa kiểm chứng"** thay vì suy diễn.

2. Bài toán, mục tiêu và phạm vi

2.1. Bối cảnh

Trong các phiên livestream, bình luận của người xem đến liên tục và có thể thay đổi rất nhanh. Một đợt bùng phát bình luận tiêu cực — do tranh cãi, hiểu lầm, hoặc bị kích động — có thể làm hỏng không khí cộng đồng và gây áp lực lên người sáng tạo nội dung. Kiểm duyệt thủ công không theo kịp tốc độ này: người trực chỉ có thể đọc một phần rất nhỏ các bình luận, và thường chỉ phát hiện vấn đề khi nó đã lan rộng.

Một hệ thống tự động không thay thế người kiểm duyệt, nhưng có thể **phát hiện tín hiệu sớm**: tỷ lệ bình luận tiêu cực tăng bất thường trong một cửa sổ thời gian ngắn. Đó là bài toán mà SentimentGuard hướng tới.

2.2. Phát biểu bài toán

Đầu vào: mỗi bản ghi là một bình luận đã được làm sạch, gồm các trường `comment_id`, `stream_id`, `user_hash`, `text`, `event_ts`. Trường `user_hash` là mã băm có muối, không chứa danh tính thật.

Đầu ra: một nhãn cảm xúc thuộc tập {`negative`, `neutral`, `positive`} kèm xác suất của lớp được chọn (với các mô hình có hỗ trợ xác suất). Kết quả được ghi vào Kafka topic `comments_scored` và bảng chi tiết trong Parquet.

Bài toán tổng hợp: với mỗi cửa sổ thời gian (1 phút hoặc 10 giây trượt), tính số bình luận, số bình luận mỗi lớp, tỷ lệ tiêu cực và độ trễ trung bình. Nếu tỷ lệ tiêu cực vượt 30% **và** cửa sổ có ít nhất 5 bình luận, dashboard bật cảnh báo.

Ràng buộc: không đưa nhãn vào luồng streaming; nhãn chỉ dùng để đánh giá và được lưu riêng trong `data/processed/eval_labels.parquet`, ghép lại theo `comment_id`. Dữ liệu streaming là dữ liệu phát lại, không phải dữ liệu thật.

2.3. Mục tiêu của đồ án

1. **Xây dựng đường ống dữ liệu tái lập được:** từ file JSON gốc của IMDb đến ba tập train/validation/test cố định, có nhật ký làm sạch và kiểm tra rò rỉ dữ liệu.
2. **Huấn luyện và so sánh nhiều mô hình:** trên cùng một pipeline TF-IDF, chọn mô hình bằng macro-F1 trên validation, chỉ đánh giá test một lần.
3. **Xây dựng luồng streaming thực sự:** Kafka + Spark Structured Streaming, schema tường minh, event time, watermark, cửa sổ thời gian và DLQ.
4. **Kiểm chứng tính nhất quán:** so sánh dự đoán trên luồng với dự đoán offline trên cùng `comment_id`, đối chiếu số bản ghi gửi/nhận.
5. **Cung cấp dashboard và cảnh báo:** Streamlit đọc Parquet, hiển thị tình trạng trực tiếp, cho phép thử mô hình và giải thích giới hạn.
6. **Bảo đảm quyền riêng tư và tài liệu hóa:** băm có muối mã người dùng, không lưu văn bản mặc định, tài liệu hóa từng bước để người khác chạy lại.

2.4. Phi mục tiêu (những gì đồ án không làm)

- **Không định danh hay theo dõi cá nhân.** Hệ thống chỉ giữ mã băm có muối; không lưu tên, email, số điện thoại hay ID thật.
- **Không tự động xóa, chặn hay báo cáo bình luận.** Đầu ra chỉ là tín hiệu để người kiểm duyệt xem lại.

- **Không phải hệ thống sản xuất.** Không có cam kết SLA, không có giám sát 24/7, không có cơ chế chịu lỗi đa máy.
- **Không huấn luyện trên bình luận livestream thật.** Toàn bộ số liệu được đo trên review IMDb công khai (CC0); phần streaming là replay.
- **Không tuyên bố exactly-once.** Chỉ khẳng định at-least-once cho phần đã kiểm chứng bằng thực nghiệm.

2.5. Tiêu chí Technology for Good

Đề tài hướng tới lợi ích cộng đồng: giúp người sáng tạo nội dung và cộng đồng phát hiện sớm bình luận độc hại, giảm tác động tiêu cực đến người tham gia. Hệ thống không phục vụ mục đích thương mại đơn lẻ, không dự đoán giá hay cờ bạc, và không chỉ nhằm chứng minh kỹ thuật. Thiết kế quyền riêng tư được đặt ngang hàng với chất lượng mô hình.

Tuy nhiên, cần thẳng thắn: một bộ phân loại cảm xúc có thể bị lạm dụng để kiểm duyệt quá mức. Vì vậy đồ án chủ động **không tự động chặn**, chỉ đưa ra cảnh báo tổng hợp, và ghi rõ giới hạn của mô hình trong tài liệu.

2.6. Câu hỏi kỹ thuật trọng tâm

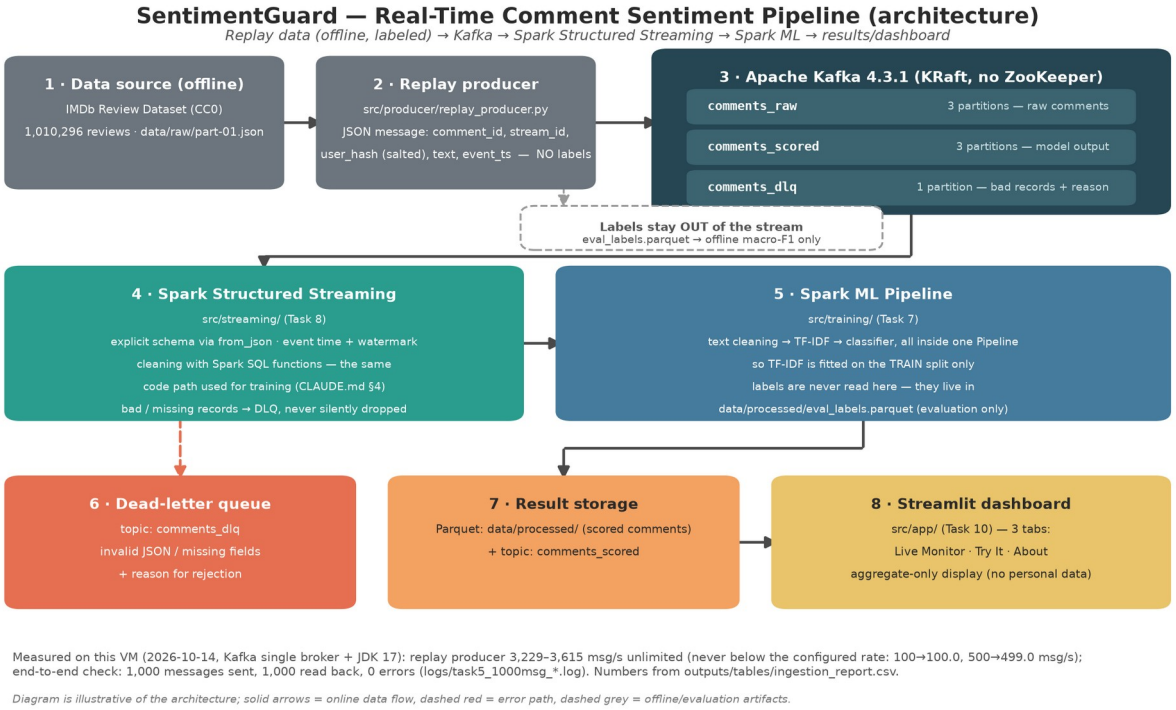
Đồ án được thiết kế để trả lời năm câu hỏi, mỗi câu đều có bằng chứng thực nghiệm tương ứng:

Bảng 2. Câu hỏi kỹ thuật và bằng chứng

Câu hỏi	Bằng chứng / nơi trả lời
Mô hình đa lớp đạt mức nào trên dữ liệu mất cân bằng?	Chương 6 — macro-F1 và F1 từng lớp trên test
TF-IDF có bị rò rỉ từ tập test không?	Chương 4 — TF-IDF nằm trong Pipeline, leak_check = 0
Luồng streaming có cho cùng dự đoán với offline?	Chương 9 — 1.000/1.000 dự đoán trùng khớp
Bản ghi lỗi có bị bỏ âm thầm không?	Chương 8 — DLQ kèm lý do, đối chiếu 1.014 + 10 = 1.024
Hệ thống chịu được tải nào, độ trễ bao nhiêu?	Chương 9 — đỉnh 384 msg/s, p50 2,4 s, p95 9,7 s

3. Kiến trúc hệ thống và luồng dữ liệu

Hệ thống được tổ chức thành năm tầng độc lập nhưng dùng chung cấu hình và dùng chung hàm làm sạch văn bản. Sơ đồ dưới đây là hình kiến trúc chính thức của đồ án (`outputs/figures/00_pipeline_architecture.png`).



Hình 1. Kiến trúc tổng thể của SentimentGuard: dữ liệu → huấn luyện → streaming → dashboard.

3.1. Các thành phần

Bảng 3. Thành phần chính và vai trò

Tầng	Thành phần	Vai trò	Công nghệ
Nguồn	IMDb Review Dataset - ebD	Review công khai CC0, dùng làm proxy cho bình luận	JSON 1,09 GB
Chuẩn bị	src/training/prepare.py	Làm sạch, gán nhãn, loại trùng, chia tập	PySpark 3.5.9
Huấn luyện	src/training/train.py	TF-IDF + so sánh 4 mô hình, lưu PipelineModel	Spark ML
Mô phỏng	src/producer/replay_producer.py	Phát lại bình luận vào Kafka theo tốc độ cấu hình	kafka-python
Truyền tải	Kafka 4.3.1 (KRaft)	Topic comments_raw / comments_scored / comments_dlq	Kafka
Streaming	src/streaming/stream_job.py	6 query: chấm điểm, ghi Kafka, cửa sổ, DLQ	Structured Streaming
Lưu trữ	Parquet trên ổ đĩa cục bộ	Bảng chi tiết, cửa sổ, DLQ, batch log	Parquet
Hiển thị	src/app/app.py	Dashboard 3 tab + quy tắc cảnh báo	Streamlit
Thử nghiệm	src/app/model_trial.py	Chấm câu tự nhập / mẫu test, không cần Kafka	PySpark local

3.2. Luồng dữ liệu huấn luyện (batch)

1. `prepare.py` đọc file JSON gốc, cắt `review_detail` ở 20.000 ký tự và giữ một tập con tất định theo `crc32(review_id)`.
2. Văn bản được làm sạch bằng `src/common/text_cleaning.py` — cùng hàm sẽ được dùng lại trong streaming.

3. Nhãn được suy từ rating IMDb: 1–4 → negative, 5–6 → neutral, 7–10 → positive.
4. Các bản ghi thiếu văn bản, thiếu nhãn, quá ngắn, quá dài, trùng mã, trùng chính xác và gần trùng đều bị loại; nhật ký ghi vào `cleaning_log.csv`.
5. Chia train/validation/test theo tỷ lệ 70/15/15, cân bằng theo nhóm phim (`stream_id`).
6. `train.py` dựng Pipeline TF-IDF, chạy `TrainValidationSplit` bên trong train để chọn siêu tham số, đánh giá trên validation, rồi lưu mô hình.
7. Sau khi chọn xong mô hình tốt nhất, tập test được dùng **một lần duy nhất** để báo cáo kết quả cuối cùng.

3.3. Luồng dữ liệu streaming (replay)

1. `replay_producer.py` đọc các bản ghi từ tập test (không gửi nhãn), tạo message JSON gồm đúng 5 trường và gửi vào topic `comments_raw`.
2. Job Spark đọc Kafka với `startingOffsets=latest`, schema khai báo tường minh, parse JSON ở chế độ PERMISSIVE và ghi lại bản ghi hỏng.
3. Mỗi bản ghi được làm sạch bằng đúng hàm dùng lúc huấn luyện. Bản ghi không hợp lệ được gán `dlq_reason` và chuyển sang DLQ.
4. Bản ghi hợp lệ được đưa qua PipelineModel đã lưu để dự đoán nhãn và xác suất; kết quả ghi ra Kafka `comments_scored` và bảng Parquet chi tiết.
5. Hai query cửa sổ (1 phút và 10 giây trượt) tổng hợp số lượng theo lớp, tỷ lệ và độ trễ trung bình.
6. Dashboard Streamlit đọc Parquet, hiển thị bảng, biểu đồ, cảnh báo và cho phép thử mô hình với câu tự nhập.

3.4. Các quyết định thiết kế quan trọng

Bảng 4. Quyết định thiết kế và lý do

Quyết định	Lý do
Làm sạch văn bản chỉ viết một lần	Tránh lệch phân phối giữa lúc train và lúc chạy thật; cùng một hàm Spark SQL được gọi ở cả hai phía.
TF-IDF nằm trong Pipeline	Bảo đảm vectorizer chỉ fit trên train, không học từ validation/test.
Schema tường minh khi from_json	Không suy luận schema từ dữ liệu; kiểm soát kiểu và trường bắt buộc.
Event time + watermark 2 phút	Cửa sổ phản ánh thời điểm bình luận xảy ra, không phải thời điểm máy xử lý; watermark chặn state tăng vô hạn.
Mỗi query một checkpoint riêng	Tránh xung đột offset và state giữa 6 query.
DLQ thay vì bỏ qua	Bản ghi lỗi được giữ kèm lý do để điều tra, không biến mất âm thầm.
Cấu hình trong config.yaml	Không hard-code đường dẫn, ngưỡng hay tham số; dễ tái lập và thay đổi.
Seed 42 cho mọi bước ngẫu nhiên	Tái lập kết quả giữa các lần chạy.

4. Dữ liệu: nguồn, làm sạch và chia tập

4.1. Nguồn dữ liệu

Đồ án dùng **IMDb Review Dataset - ebD** — bộ review phim công khai, giấy phép **CC0**, gồm 1.010.293 review trong file `part-01.json` có kích thước khoảng 1,09 GB. Mỗi review có điểm rating 1–10, nội dung văn bản, mã phim, mã người đánh giá và một số trường phụ.

Vì máy ảo không đủ tài nguyên để huấn luyện trên toàn bộ 1,01 triệu review, đồ án giữ **201.074 bản ghi (19,9%)** bằng cách chọn tất định theo `crc32(review_id)`. Cách chọn này trải đều trên toàn file, không dồn về đầu file, và chạy lại trên cùng dữ liệu sẽ ra đúng tập cũ. File manifest `data/processed/imdb_jsonl/_manifest.json` ghi lại nguồn, kích thước, cách lấy mẫu và số bản ghi.

> **Lưu ý tái lập quan trọng**
> Tập 201.074 bản ghi hiện có được tạo bằng code cũ với điều kiện `crc32(review_id) % 5 == 0` (manifest ghi `subsample_mod: 5`). `config.yaml` hiện tại đã đổi sang `prepare.subsample_pct: 64`, dùng dải tiền tố `crc32 % 1000 < 640` — **một tập con khác**, không phải tập cũ mở rộng. Vì vậy muốn tái lập đúng các số liệu trong báo cáo, phải dùng dữ liệu đã xử lý trong `data/processed/` hoặc giữ nguyên JSONL/manifest; chạy lại `prepare.py` từ raw với config 64% sẽ ra tập khác.

Bảng 5. Các trường dữ liệu nguồn và cách dùng

Trường nguồn	Kiểu	Dùng trong đồ án	Ghi chú
review_id	chuỗi	→ comment_id	Không duy nhất trong nguồn
reviewer	chuỗi	→ user_hash	Băm có muối, không lưu tên
movie	chuỗi	→ stream_id	Dùng để chia tập cân bằng theo phim
rating	chuỗi 1–10	→ label	Nguồn của nhãn; thiếu 5–8%
review_detail	chuỗi	→ text	Cắt tối đa 20.000 ký tự
review_date	chuỗi	Giữ tham khảo	Không dùng làm đặc trưng
review_summary	chuỗi	Không dùng	Tách riêng khỏi nội dung
spoiler_tag	bool	Không dùng	Không liên quan cảm xúc
helpful	mảng	Không dùng	Không liên quan cảm xúc

4.2. Nhãn và vấn đề nhãn proxy

Nhãn không được gán tay mà **suy ra từ rating**: 1–4 là negative, 5–6 là neutral, 7–10 là positive. Cách này cho phép tận dụng dữ liệu lớn có sẵn, nhưng có hai hệ quả cần nói rõ:

- **Nhãn neutral rất nhiễu.** Một review rating 5–6 có thể khen chê lẫn lộn, hoặc chỉ là "xem được". Mô hình không có tín hiệu văn bản đủ mạnh để phân biệt neutral với hai lớp còn lại — đây là lý do F1 neutral chỉ 0,378.
- **Nhãn là proxy, không phải cảm xúc thật.** Một review có thể mỉa mai, hoặc dùng từ "bad" để khen ("so bad it's good"). Báo cáo không được coi nhãn IMDb là chân lý.

Vì vậy, mọi chỉ số trong đồ án được hiểu là **chỉ số trên dữ liệu IMDb**, không phải chỉ số trên bình luận livestream thật. Khi chuyển sang dữ liệu thật, cần gán nhãn tay một mẫu và đo lại trước khi tin vào con số.

4.3. Làm sạch văn bản

Hàm làm sạch nằm duy nhất ở `src/common/text_cleaning.py` và được dùng cho cả huấn luyện lẫn streaming. Toàn bộ phép biến đổi là **hàm Spark SQL** (`lower`, `regexp_replace`, `trim`, `split`...), không dùng Python UDF, nên không chặn tối ưu hóa của Catalyst và chạy được trong executor JVM. Thứ tự xử lý có ý nghĩa — đổi thứ tự sẽ đổi kết quả:

1. Chuyển toàn bộ về chữ thường và trim hai đầu.
2. Bỏ thẻ HTML và entity (`&`, `"`, `'`...). Bước này làm **trước** khi chèn token để regex không ăn mất token của chính mình.
3. Thay mọi URL bằng token `<url>`.
4. Thay @mention bằng token `<user>` — không giữ danh tính.
5. Chuẩn hóa dấu câu Unicode (’ ‘ “ ” – —) về ASCII và bỏ ký tự vô hình.
6. Thay viết tắt/teencode theo từ điển trong `config.yaml`, theo ranh giới từ.
7. Rút gọn ký tự lặp từ 3 lần trở lên thành 2: `haaaay` → `haay`.
8. Tách emoji thành token riêng để đếm được tần suất.
9. Gộp mọi khoảng trắng liên tiếp thành một và trim.

Hàm có tính **idempotent**: chạy hai lần cho cùng kết quả. Điều này được kiểm tra bằng test tự động, cùng với các trường hợp HTML, URL, mention, emoji và chuỗi rỗng.

4.4. Nhật ký làm sạch và loại trùng

Bảng 6. Nhật ký làm sạch (outputs/tables/cleaning_log.csv)

Bước	Số bản ghi bị loại	Số còn lại	Ghi chú
01 — Nạp raw	—	201.074	Từ JSONL, đã áp dụng mẫu
02 — Thiếu văn bản	0	201.074	Không có bản ghi rỗng
03 — Thiếu nhãn	10.394	190.680	Rating thiếu hoặc ngoài 1–10
04 — Quá ngắn	10	190.670	Dưới ngưỡng min_chars
05 — Quá dài	1.686	188.984	Trên ngưỡng max_chars
06 — Trùng mã	2.999	185.985	Cùng comment_id
07 — Trùng chính xác	636	185.349	Cùng văn bản sau chuẩn hóa
08 — Gần trùng	63	185.286	Tương đồng cao sau chuẩn hóa
09 — Còn lại	—	185.286	Dùng để chia tập

Việc loại trùng **trước khi chia tập** là bắt buộc với dữ liệu bình luận: người dùng thường copy-paste cùng một câu, và nếu bản copy nằm ở cả train lẫn test thì điểm sẽ cao giả tạo. Sau khi chia, `train.py` chạy thêm một kiểm tra rò rỉ trên văn bản đã chuẩn hóa; kết quả trong `outputs/tables/leak_check.json` là **0** cho cả ba cặp train↔val, train↔test, val↔test.

4.5. Chia tập

Dữ liệu được chia **70% train / 15% validation / 15% test**, phân tầng theo nhóm phim (`stream_id`) để ba tập có cùng tỷ lệ lớp và không có phim nào chỉ xuất hiện ở một tập. Tập test được dùng đúng một lần vào cuối cùng.

Bảng 7. Phân bố lớp theo tập (outputs/tables/class_distribution.csv)

Tập	Negative	Neutral	Positive	Tổng
Train	32.504 (25,06%)	17.983 (13,86%)	79.214 (61,07%)	129.701
Validation	6.965 (25,06%)	3.854 (13,87%)	16.974 (61,07%)	27.793
Test	6.965 (25,06%)	3.853 (13,86%)	16.974 (61,07%)	27.792

Tỷ lệ ba lớp gần như giống hệt nhau giữa ba tập (chênh dưới 0,02 điểm phần trăm). Điều này cho phép so sánh validation và test mà không lo lệch phân phối. Tuy nhiên, dữ liệu vẫn **mất cân bằng rõ rệt**: positive chiếm 61%, neutral chỉ 14%. Vì vậy accuracy không phải chỉ tiêu đủ tốt — phải báo cáo macro-F1 và F1 từng lớp.

4.6. Quyền riêng tư

- `user_hash = sha256(salt + reviewer)[:16]`; muối đọc từ biến môi trường `COMMENT_HASH_SALT`, không nằm trong code hay config.
- Không lưu tên, email, số điện thoại, ID thật hay bất kỳ trường định danh nào.
- Khi chạy streaming thật, `privacy.keep_text: false` — văn bản không được ghi vào sink. Cờ `--keep-text` chỉ dùng cho dữ liệu demo công khai (IMDb CC0).
- Dashboard chỉ hiển thị kết quả tổng hợp; không hiển thị danh tính người dùng.

4.7. Hạn chế của dữ liệu

Có ba hạn chế cần ghi nhớ. Thứ nhất, **không phải review nào cũng có rating đầy đủ** — 10.394 bản ghi bị loại ở bước 03. Thứ hai, **cùng một người có thể xuất hiện ở nhiều tập**: 11.427 trong 111.087 người dùng (10,3%) có mặt ở hơn một tập, tương ứng 75.939 dòng (41,0%). Đây là hạn chế đã biết; đồ án chọn chia theo phim thay vì chia theo người, nên không loại bỏ hoàn toàn hiện tượng này. Thứ ba, **dữ liệu IMDb là review phim, không phải bình luận livestream**: câu dài hơn, có điểm số, ít teencode và không có tiếng Việt. Mọi kết luận về hành vi livestream chỉ là suy luận có điều kiện.

Bảng 8. Kiểm tra chồng lấn người dùng giữa các tập

Chỉ số	Giá trị
Tổng số người dùng (mã băm)	111.087
Người dùng xuất hiện ở hơn một tập	11.427 (10,3%)
Số dòng thuộc người dùng chồng lấn	75.939 (41,0% của 185.286)
Trùng văn bản train↔val	0
Trùng văn bản train↔test	0
Trùng văn bản val↔test	0

5. Mô hình và quy trình huấn luyện

5.1. Công thức bài toán

Với mỗi bình luận có văn bản đã làm sạch `x`, mô hình học ánh xạ `f(x) → y`, trong đó `y ∈ {0, 1, 2}` tương ứng negative, neutral, positive. Mô hình được huấn luyện trên tập train, chọn siêu tham số bằng một phần của train, đánh giá trên validation, và cuối cùng đo trên test.

Vì dữ liệu mất cân bằng, hàm mục tiêu không chỉ là accuracy. Trong quá trình chọn mô hình, đồ án dùng **weighted F1** cho `TrainValidationSplit` (để không bỏ qua lớp thiểu số nhưng vẫn phản ánh độ chính xác tổng thể), sau đó so sánh các mô hình trên validation bằng **macro-F1** — chỉ số đối xử bình đẳng giữa ba lớp.

5.2. Pipeline TF-IDF

Toàn bộ quá trình biến đổi văn bản thành vector nằm trong một Spark ML Pipeline. Điều này bảo đảm **vectorizer chỉ được fit trên train**: khi `Pipeline.fit(train)` chạy, các stage học từ vựng và IDF chỉ nhìn thấy train; validation và test chỉ được `transform`, không được học.

Bảng 9. Các stage trong Pipeline

Stage	Đầu vào → đầu ra	Tham số chính	Vai trò
RegexTokenizer	text → tokens	pattern `\s+`	Tách theo khoảng trắng; văn bản đã lower ở bước làm sạch
StopWordsRemover	tokens → words	162 stopword, giữ phủ định	Bỏ từ chức năng nhưng giữ not/no/never…
NGram	words → ngrams	n = 1	Unigram; n = 2 cũng đã thử với NB và LR
CountVectorizer	ngrams → tf	vocab 20.000, minDF 3	Đếm tần suất, giới hạn từ vựng
IDF	tf → tfidf	—	Giảm trọng số từ xuất hiện ở nhiều văn bản
Normalizer	tfidf → features	L2	Chuẩn hóa vector để độ dài không chi phối
Classifier	features → prediction	Theo từng mô hình	LogisticRegression / NaiveBayes / LinearSVC / RandomForest

5.3. Tách từ, stopword và phủ định

Văn bản đã được làm sạch và chuyển về chữ thường, nên `RegexTokenizer` chỉ cần tách theo khoảng trắng (`pattern = "\\s+"`). Cách này đơn giản và nhanh, nhưng có một hạn chế đã đo được: **11,2% token còn dính dấu câu ở hai đầu** (`movie.` khác `movie`), làm từ vựng phình từ 318.980 lên 524.107 từ (+64%) khi so sánh với cách tách tốt hơn. Đồ án chưa sửa vì sửa là phải huấn luyện lại toàn bộ mô hình mới so sánh được.

StopWordsRemover dùng danh sách mặc định tiếng Anh của Spark (162 từ sau khi thêm từ tùy chỉnh). Điểm quan trọng: **các từ phủ định được giữ lại** — `not`, `no`, `never`, `cannot`, `don't`, `isn't`... Nếu bỏ chúng, câu “not bad” sẽ thành “bad” và mô hình hiểu ngược nghĩa. Tuy vậy, vì `n = 1`, mô hình vẫn không thấy được quan hệ giữa “not” và “bad”; đây là một nguyên nhân chính gây lỗi (xem chương 7).

5.4. TF-IDF và giới hạn từ vựng

`CountVectorizer` giữ tối đa **20.000 từ** có tần suất tài liệu ít nhất là **3** (`minDF = 3`). Giới hạn này vừa kiểm soát bộ nhớ, vừa giảm nhiễu từ các từ chỉ xuất hiện một lần. `IDF` giảm trọng số các từ phổ biến, còn `Normalizer` chuẩn hóa L2 để bình luận dài không tự động có vector lớn hơn.

Với 20.000 đặc trưng và 129.701 dòng train, ma trận TF-IDF là thưa. Đây là lý do các mô hình tuyến tính (LogisticRegression, LinearSVC) phù hợp hơn RandomForest — cây quyết định phải tạo nhiều phép chia trên không gian thưa và cho kết quả kém hơn rõ rệt (xem chương 6).

5.5. Trọng số lớp

Mỗi lớp được gán trọng số `w_c = N / (k × n_c)`, trong đó `N` là tổng số dòng train, `k = 3` là số lớp, `n_c` là số dòng của lớp `c`. Trọng số được tính **chỉ trên tập train** và truyền vào mô hình qua `weightCol`.

Bảng 10. Trọng số lớp tính trên tập train

Lớp	Số dòng train	Tỷ lệ	Trọng số
Negative	32.504	25,06%	1,330
Neutral	17.983	13,86%	2,404
Positive	79.214	61,07%	0,546

Ý nghĩa: mỗi lỗi ở lớp neutral bị phạt nặng gấp khoảng 1,8 lần lỗi ở lớp positive, vì neutral chỉ có 17.983 mẫu so với 79.214 của positive. Nếu không có trọng số, mô hình có xu hướng đoán gần như mọi thứ là positive và đạt accuracy 61% một cách vô nghĩa — đúng bằng baseline đoán “positive”.

5.6. Chọn siêu tham số và đánh giá

Với mỗi họ mô hình, `TrainValidationSplit` chia **80% train để fit** và **20% train để chọn siêu tham số**, thử toàn bộ tổ hợp trong lưới, chọn tổ hợp có weighted F1 cao nhất, rồi **fit lại trên toàn bộ train**. Sau đó mô hình được đánh giá trên validation. Chỉ sau khi chọn xong mô hình tốt nhất theo macro-F1 trên validation, tập test mới được dùng một lần.

Bảng 11. Lưới siêu tham số (config.yaml)

Mô hình	Tham số	Số tổ hợp
NaiveBayes	ngram ∈ {1, 2}; smoothing = 1,0	2
LogisticRegression	ngram ∈ {1, 2}; regParam ∈ {0,01; 0,1}; maxIter = 20; elasticNet = 0	4
LinearSVC (One-vs-Rest)	ngram = 1; regParam ∈ {0,01; 0,1}; maxIter = 20	2
RandomForest	ngram = 1; vocab ∈ {5.000}; numTrees = 30; maxDepth ∈ {8, 12}	2

Kết quả ghi lại cho thấy với cả NB và LR, tổ hợp được chọn đều là **n = 1**; bigram không cải thiện weighted F1 trên tập chọn. Điều này không có nghĩa bigram vô dụng, mà là với 20.000 đặc trưng và dữ liệu hiện tại, unigram đã đủ và ít nhiễu hơn. RandomForest chỉ dùng 5.000 đặc trưng để giữ thời gian huấn luyện trong giới hạn máy ảo.

5.7. Vì sao chọn LogisticRegression

- **Hiệu quả cao nhất trên validation:** macro-F1 0,629, cao hơn NaiveBayes (0,619), LinearSVC (0,622) và RandomForest (0,534).
- **Có xác suất dự đoán** — cần cho dashboard và cho phép người dùng thấy độ tin cậy; LinearSVC không có xác suất trực tiếp.
- **Diễn giải được:** trọng số của từng từ có thể đọc trực tiếp, giúp phân tích lỗi và giải thích cho hội đồng.
- **Huấn luyện nhanh, dự đoán nhanh:** 245 giây cho 4 tổ hợp trên 129.701 dòng; dự đoán khoảng 0,92 giây cho 10.000 dòng.
- **Phù hợp với đặc trưng thưa nhiều chiều:** mô hình tuyến tính thường mạnh hơn cây trên TF-IDF văn bản.

5.8. Artifact và khả năng tái lập

Mô hình được lưu tại `outputs/models/sentiment_pipeline`, kèm metadata `sentiment_pipeline_metadata.json` ghi phiên bản Spark (3.5.9), seed (42), ánh xạ nhãn, tham số tốt nhất, cấu hình vectorizer và kích thước ba tập. Pipeline gồm 7 stage: RegexTokenizer → StopWordsRemover → NGram → CountVectorizerModel → IDFModel → Normalizer → LogisticRegressionModel.

Thời gian huấn luyện đầy đủ khoảng **13 phút** cho 130.000 dòng trên máy ảo với `local[2]`. Nếu chạy lại với cấu hình 64% hiện tại, cần tăng bộ nhớ driver (`--driver-memory 4g`) và thời gian sẽ lâu hơn đáng kể.

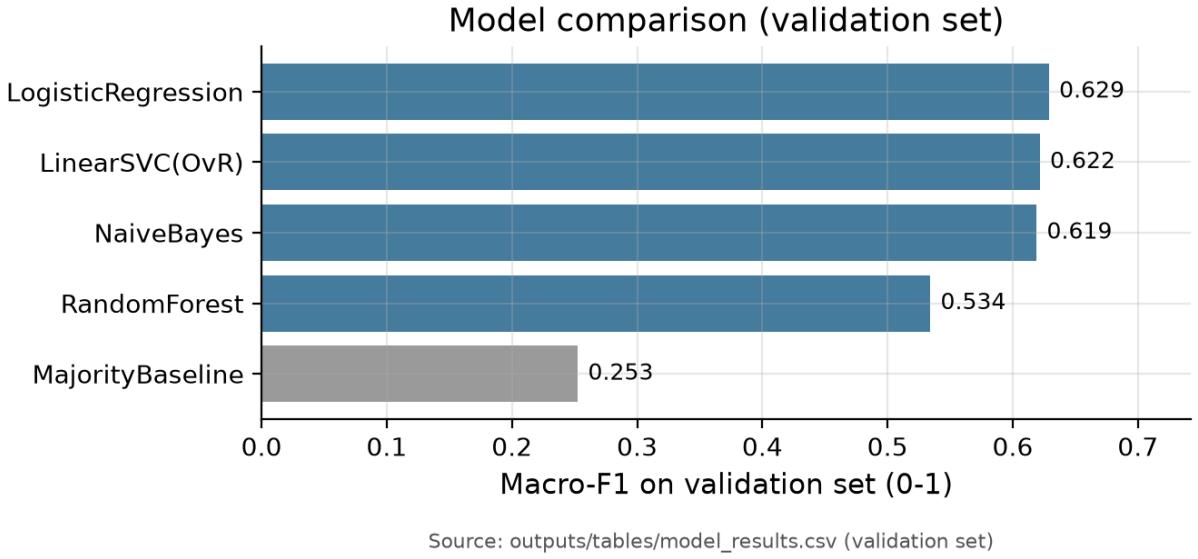
```
# Cấu hình tối thiểu để tái lập mô hình
source scripts/env.sh
$VENV_PY src/training/prepare.py
$VENV_PY src/training/train.py --models lr nb svc rf
```

6. Kết quả đánh giá mô hình

6.1. So sánh trên validation

Bảng 12. Kết quả trên validation — 27.793 dòng (model_results.csv)

Mô hình	Accuracy	Weighted-F1	Macro-F1	Macro-Precision	Macro-Recall	Thời gian train
MajorityBaseline	0,6107	0,4631	0,2528	0,2036	0,3333	0 s
NaiveBayes	0,6743	0,7013	0,6192	0,6227	0,6613	107,9 s
LogisticRegression	0,7063	0,7241	0,6291	0,6256	0,6497	245,0 s
LinearSVC (OvR)	0,7000	0,7172	0,6218	0,6174	0,6417	116,3 s
RandomForest	0,5878	0,6131	0,5343	0,5397	0,5741	282,4 s



Hình 2. So sánh macro-F1 và weighted-F1 của bốn mô hình trên validation.

Điểm đáng chú ý: **MajorityBaseline đạt accuracy 61,07%** chỉ bằng cách đoán tất cả là positive. Nếu báo cáo accuracy mà không so với baseline, con số 70,18% của mô hình trông "cao" nhưng thực chất chỉ hơn baseline khoảng 9 điểm phần trăm. Ngược lại, **macro-F1 của baseline chỉ 0,253**, còn mô hình đạt 0,623 — mức cải thiện thực sự nằm ở khả năng phân biệt ba lớp, không phải ở accuracy.

RandomForest kém hơn rõ rệt (macro-F1 0,534) dù thời gian huấn luyện dài nhất. Nguyên nhân phù hợp với lý thuyết: TF-IDF 20.000 chiều rất thưa, cây quyết định tạo các phép chia cứng và không tận dụng được cấu trúc tuyến tính của dữ liệu văn bản. LinearSVC gần bằng LogisticRegression nhưng không có xác suất, nên không được chọn.

6.2. Kết quả cuối cùng trên test

Bảng 13. Kết quả LogisticRegression trên test — 27.792 dòng

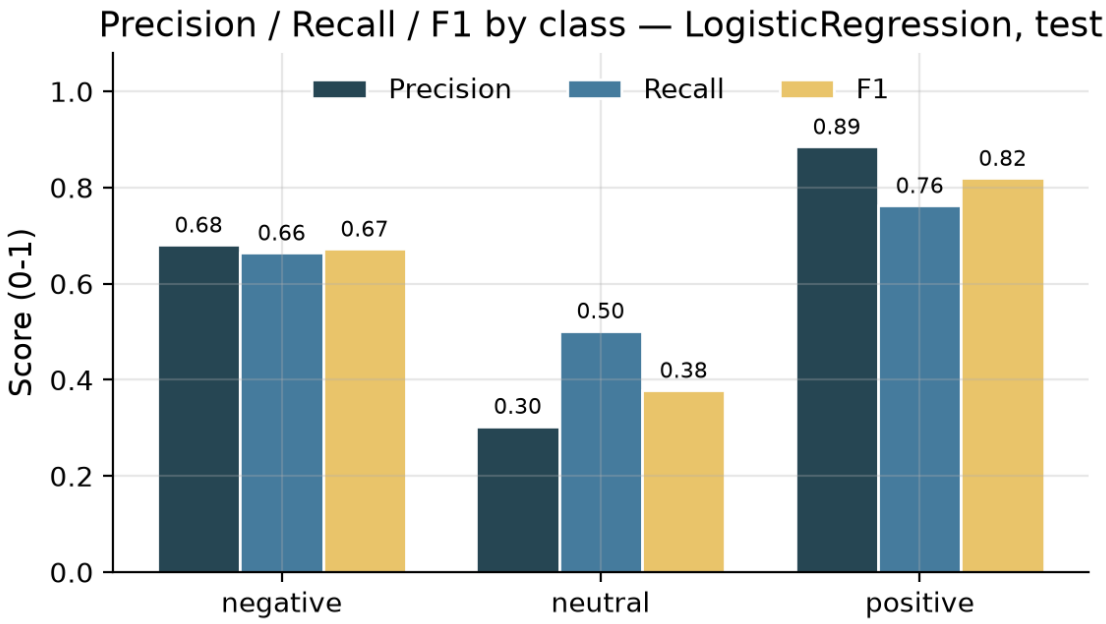
Chỉ số	Giá trị
Accuracy	0,7018 (70,18%)
Weighted-F1	0,7213 (72,13%)
Macro-F1	0,6233 (62,33%)
Macro-Precision	0,6231
Macro-Recall	0,6429
Số dòng test	27.792

So với validation, các chỉ số gần như không đổi (macro-F1 0,6291 → 0,6233), cho thấy mô hình không bị overfit vào validation và kết quả test là đáng tin trong phạm vi dữ liệu IMDb. Tuy nhiên, đây vẫn là **ước lượng trên một phân phối cụ thể**, không phải cam kết cho dữ liệu livestream.

6.3. Chỉ số theo từng lớp

Bảng 14. Precision / Recall / F1 từng lớp trên test (per_class_metrics.csv)

Lớp	Precision	Recall	F1	Support
Negative	0,6807	0,6642	0,6723	6.965
Neutral	0,3036	0,5019	0,3783	3.853
Positive	0,8851	0,7626	0,8193	16.974



Source: outputs/tables/per_class_metrics.csv — LogisticRegression on test

Hình 3. Precision, recall và F1 từng lớp trên tập test.

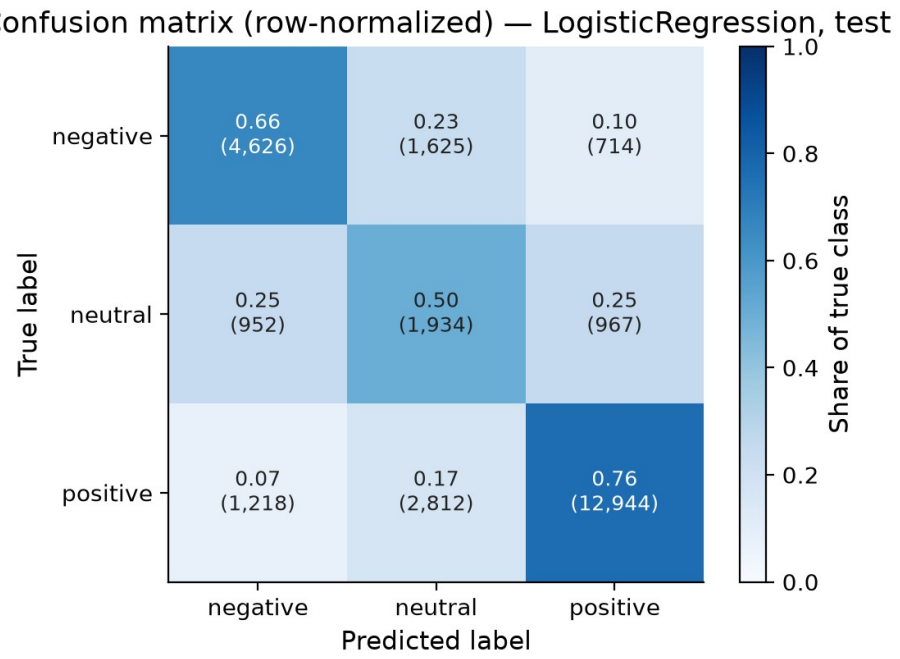
Positive là lớp mạnh nhất (F1 0,819): mô hình nhận ra lời khen rất tốt, precision 0,885 nghĩa là khi mô hình nói "positive" thì gần 9/10 lần đúng. **Negative** ở mức khá (F1 0,672), recall 0,664. **Neutral** là điểm yếu lớn nhất: precision chỉ 0,304 — khi mô hình nói "neutral", hơn 2/3 trường hợp là sai; recall

0,502 nghĩa là chỉ một nửa số neutral thật được tìm ra. Đây không phải lỗi huấn luyện đơn thuần mà là hệ quả của nhãn proxy nhiễu.

6.4. Ma trận nhầm lẫn

Bảng 15. Ma trận nhầm lẫn trên test (confusion_matrix.csv)

Thực tế \ Dự đoán	Negative	Neutral	Positive
Negative (6.965)	4.626	1.625	714
Neutral (3.853)	952	1.934	967
Positive (16.974)	1.218	2.812	12.944



Source: outputs/tables/confusion_matrix.csv — LogisticRegression on test (Task 7)

Hình 4. Ma trận nhầm lẫn của LogisticRegression trên tập test.

Đọc ma trận: **neutral bị phân tán sang cả hai phía** — 952 dòng bị đoán thành negative và 967 dòng thành positive, chỉ 1.934/3.853 đúng. Positive cũng bị kéo về neutral khá nhiều (2.812 dòng). Negative có 1.625 dòng bị đoán thành neutral, nhưng chỉ 714 dòng thành positive. Có thể thấy mô hình có xu hướng “trung dung hóa” các bình luận có tín hiệu lẫn lộn, điều này phù hợp với dữ liệu review dài, nhiều ý khen chê đan xen.

Nếu ưu tiên phát hiện bình luận tiêu cực, có thể hạ ngưỡng quyết định cho lớp negative để tăng recall, đổi lại precision giảm. Đồ án chưa tối ưu theo hướng này vì mục tiêu là báo cáo trung thực mô hình mặc định; đây là hướng phát triển ở chương 13.

6.5. So sánh với baseline và diễn giải

Bảng 16. Mức cải thiện so với baseline đoán "positive"

Chỉ số	MajorityBaseline	LogisticRegression	Chênh lệch
Accuracy	0,6108	0,7018	+0,0910
Macro-F1	0,2528	0,6233	+0,3705
F1 negative	0,000	0,6723	+0,6723
F1 neutral	0,000	0,3783	+0,3783
F1 positive	0,758	0,8193	+0,0613

Cải thiện lớn nhất nằm ở **negative và neutral** — hai lớp mà baseline không bao giờ đoán đúng. Đây là bằng chứng mạnh nhất cho thấy mô hình học được tín hiệu thực sự, không chỉ đoán theo lớp đa số. Tuy nhiên, mức cải thiện ở positive chỉ +0,061 vì baseline vốn đã đoán đúng phần lớn positive.

Một cách đọc khác: nếu mục tiêu chỉ là "đoán đúng phần lớn", baseline 61% đã đủ. Giá trị của mô hình nằm ở việc **tách được negative và neutral**, phục vụ cảnh báo sớm. Vì vậy báo cáo dùng macro-F1 làm chỉ tiêu chính, đúng theo quy ước của đồ án.

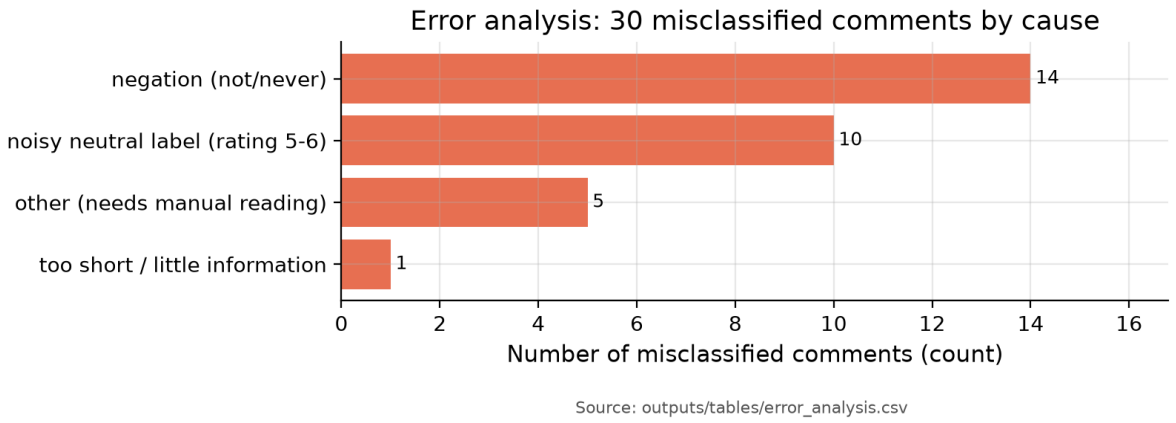
7. Phân tích lỗi và diễn giải

7.1. Phân bố lỗi

Đồ án trích 30 lỗi có xác suất cao nhất trên test vào `outputs/tables/error_analysis.csv` và phân loại thủ công. Kết quả:

Bảng 17. Phân loại 30 lỗi được phân tích

Nhóm lỗi	Số lượng	Diễn giải
Phủ định (not/never...)	14	Mô hình bỏ qua hoặc hiểu sai phạm vi phủ định
Nhãn neutral nhiễu	10	Review 5–6 điểm thường khen chê lẫn lộn, khó gọi là neutral
Khác (cần đọc tay)	5	Mỉa mai, đánh giá có điều kiện, câu phức tạp
Quá ngắn, ít thông tin	1	Không đủ ngữ cảnh để phân loại
Tổng	30	—



Hình 5. Phân bố lý do lỗi trong 30 mẫu được phân tích.

7.2. Vì sao phủ định là nhóm lỗi lớn nhất

Với TF-IDF unigram, câu “the acting is not bad” được biểu diễn bằng các từ `acting`, `not`, `bad` — mô hình thấy `bad` và có xu hướng đoán negative, dù nghĩa thực là positive. Việc giữ stopword `not` giúp mô hình biết sự có mặt của phủ định, nhưng **không đủ để học quan hệ giữa `not` và `bad`** vì hai từ là hai chiều độc lập trong vector. Đây là hạn chế cố hữu của bag-of-words, không phải lỗi cài đặt.

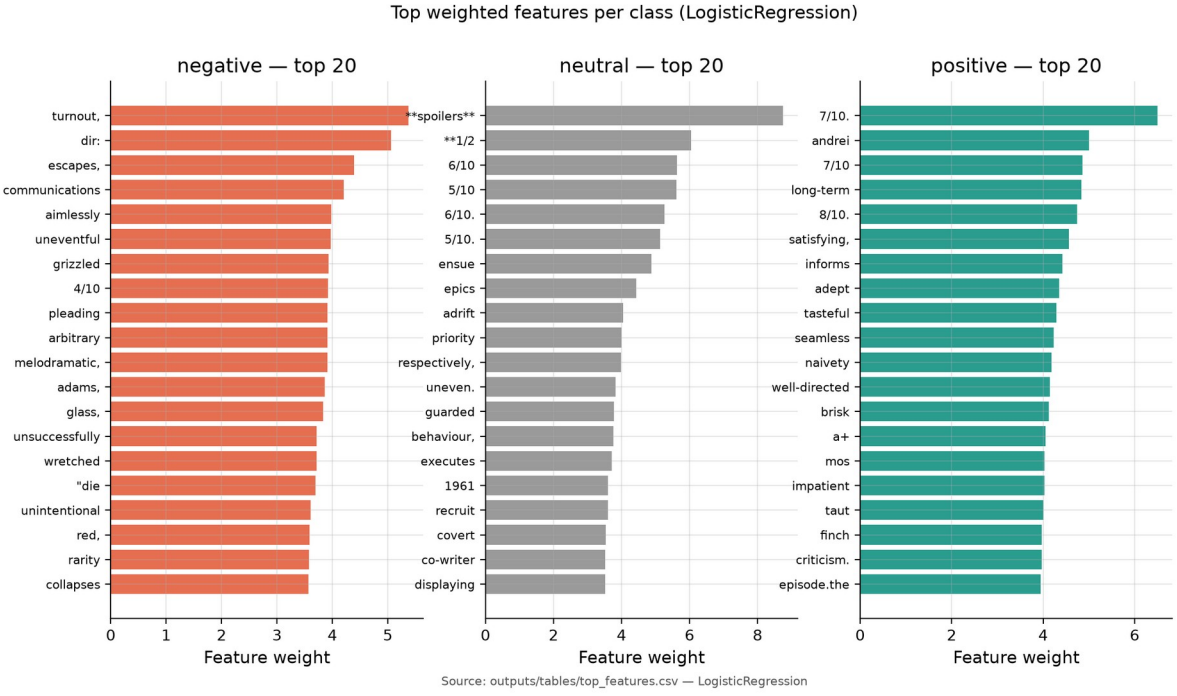
Một ví dụ thật trong `error_analysis.csv`: câu “it made us to realise what's actually important to us. i wish, this never got an end..” được gán nhãn positive (rating cao) nhưng mô hình đoán negative, rất có thể vì từ `never` và cấu trúc câu. Một ví dụ khác: “his 'bad acting' had me crying with laughter, absolutely brilliant” — từ `bad` xuất hiện nhưng câu là khen; mô hình có thể bị kéo về negative hoặc neutral.

Hướng cải thiện: dùng **bigram** cho các cặp phủ định (`not bad`, `never better`), hoặc thay bag-of-words bằng mô hình ngữ cảnh (PhoBERT cho tiếng Việt, BERT cho tiếng Anh). Đồ án đã thử bigram cho NB và LR nhưng lưới chọn n = 1; điều này cho thấy bigram đơn thuần chưa đủ, cần thiết kế đặc trưng phủ định có chủ đích hoặc mô hình ngữ cảnh.

7.3. Nhãn neutral và vấn đề “điểm số”

10/30 lỗi được xếp vào nhóm “nhãn neutral nhiễu”. Ví dụ: một review rating 5–6 có thể viết “nothing amazing, but guarantee you won't turn it off” — người đọc có thể coi là neutral hoặc hơi positive; nhãn từ rating không phản ánh sắc thái này. Mô hình không thể học một định nghĩa ổn định cho neutral nếu chính nhãn không ổn định.

Một vấn đề liên quan: **mô hình đang học điểm số trong văn bản**. Trong 20 từ đặc trưng nhất của lớp neutral, 2 từ là dạng “x/10” (“6/10”, “5/10”); trong top feature của LR, `6/10`, `5/10`, `6/10.`, `5/10.` đều có trọng số cao. Điều này có nghĩa mô hình có thể đạt F1 neutral cao hơn thực lực nếu bình luận có điểm số, và **sẽ yếu đi** trên livestream thật — nơi người xem không ghi “6/10”.



Hình 6. Trọng số lớn nhất của LogisticRegression theo từng lớp; các feature dạng điểm số nổi bật ở lớp neutral.

Vì vậy, con số macro-F1 0,6233 phải được đọc là **mức trần lạc quan** trên dữ liệu IMDb. Khi chuyển sang bình luận livestream thật, cần đo lại trên một mẫu gán nhãn tay; dự kiến chỉ số sẽ giảm, đặc biệt ở neutral.

7.4. Mỉa mai và câu phức tạp

Nhóm "khác" gồm 5 lỗi, tiêu biểu là mỉa mai hoặc khen chê đan xen. Ví dụ một review khen diễn xuất nhưng chê kịch bản, hoặc dùng "so bad it's good". Bag-of-words không nắm được ý định của người viết; đây là giới hạn của mô hình túi từ, không phải của dữ liệu. Với dữ liệu livestream, mỉa mai và teencode sẽ còn phổ biến hơn, nên cần mô hình ngữ cảnh hoặc ít nhất là character n-gram.

Tuy nhiên, cần tránh phóng đại: trong 30 lỗi được chọn, chỉ 5 lỗi thuộc nhóm này. Không nên nói "mô hình không hiểu mỉa mai" như một kết luận tổng quát; chỉ nên nói đây là một nguồn lỗi đã quan sát được, với tần suất chưa được đo trên toàn bộ tập test.

7.5. Bài học từ phân tích lỗi

- **Không tối ưu accuracy đơn thuần.** Baseline đã 61%; cải thiện thật nằm ở macro-F1 và negative/neutral.
- **Nhãn quyết định trần hiệu năng.** Neutral nhiễu đặt trần khoảng 0,38 F1; muốn vượt phải cải thiện định nghĩa nhãn hoặc dữ liệu gán tay.
- **Đặc trưng phải phù hợp miền dữ liệu.** Điểm số "x/10" giúp ích trên IMDb nhưng sẽ không tồn tại trên livestream; cần kiểm tra lại trước khi dùng.
- **Phủ định cần thiết kế có chủ đích.** Giữ stopword là chưa đủ; cần bigram/phủ định scope hoặc mô hình ngữ cảnh.
- **Luôn đọc lỗi, không chỉ nhìn chỉ số.** 30 mẫu lỗi cho nhiều thông tin hơn một bảng metric.

8. Hệ thống streaming thời gian thực

8.1. Kiến trúc Kafka và Spark

Kafka chạy ở chế độ **KRaft** (không cần ZooKeeper), phiên bản 4.3.1, với ba topic. Spark Structured Streaming đọc topic `comments_raw`, chấm điểm bằng PipelineModel đã lưu, rồi ghi kết quả ra `comments_scored` và Parquet. Bản ghi lỗi đi vào `comments_dlq` và một bảng Parquet riêng.

Bảng 18. Sáu streaming query độc lập

Query	Nguồn	Đích	Vai trò
detail	comments_raw	Parquet scored_detail	Bảng chi tiết từng bình luận
kafka	comments_raw	Kafka comments_scored	Phát kết quả cho consumer khác
window	comments_raw	Parquet window_1min	Tổng hợp cửa sổ 1 phút
sliding	comments_raw	Parquet window_10s	Tổng hợp cửa sổ 10 giây, trượt 5 giây
dlq_kafka	comments_raw	Kafka comments_dlq	Bản ghi lỗi kèm lý do
dlq_parquet	comments_raw	Parquet dlq	Lưu DLQ để điều tra

Mỗi query có `checkpointLocation` riêng dưới `checkpoints/streaming/<run>/`. Việc tách query không chỉ để rõ ràng: nếu dùng chung checkpoint, offset và state của các query sẽ giẫm lên nhau, gây lỗi hoặc mất dữ liệu. Tất cả query dùng chung một trigger 10 giây (có thể đổi bằng `--trigger-seconds`) và `maxOffsetsPerTrigger=5000`.

8.2. Schema tường minh và parse JSON

Schema được khai báo tường minh, không suy luận từ dữ liệu. Điều này quan trọng vì message Kafka có thể thiếu trường hoặc sai kiểu; nếu để Spark suy luận, schema có thể thay đổi giữa các micro-batch.

```
RAW_SCHEMA = StructType([
    StructField("comment_id", StringType(), True),
    StructField("stream_id", StringType(), True),
    StructField("user_hash", StringType(), True),
    StructField("text", StringType(), True),
    StructField("event_ts", StringType(), True),
    StructField("_corrupt", StringType(), True),
])
REQUIRED_FIELDS = ("comment_id", "stream_id", "user_hash", "text", "event_ts")
```

Một chi tiết đã gặp thật trên Spark 3.5.9: `from_json` **không tự thêm** trường chứa bản ghi hỏng dù đã đặt `columnNameOfCorruptRecord`; phải khai báo `_corrupt` trong schema thì trường này mới được điền. Nếu không, JSON hỏng sẽ mất dấu và không vào được DLQ. Đây là lý do schema có trường `_corrupt`.

8.3. Event time, watermark và cửa sổ

Mỗi bản ghi có `event_ts` — thời điểm bình luận được tạo, không phải thời điểm Kafka nhận. Spark chuyển `event_ts` thành `event_time` kiểu timestamp và dùng **watermark 2 phút** trên `event_time`. Watermark cho phép Spark đóng các cửa sổ cũ và giải phóng state, đồng thời xác định bản ghi nào được coi là "quá muộn".

Hai loại cửa sổ được dùng:

- **Cửa sổ nhảy 1 phút** (`window(event_time, '1 minute')`): tổng hợp theo từng phút, phù hợp cho báo cáo và dashboard.

- **Cửa sổ trượt 10 giây, bước 5 giây** (`window(event_time, '10 seconds', '5 seconds')`): bắt được diễn biến nhanh trong lúc sự việc đang xảy ra; các cửa sổ chồng lấn nên tổng số dòng lớn hơn số bình luận.

Công thức tổng hợp trên mỗi cửa sổ và mỗi `stream_id`: đếm tổng, đếm từng lớp, tính tỷ lệ, trung bình confidence và trung bình latency. Kết quả được ghi ra Parquet với các cột `window_start`, `window_end`, `n_comments`, `n_negative`, `n_neutral`, `n_positive`, `share_negative`, `share_neutral`, `share_positive`, `avg_confidence`, `avg_latency_ms`.

```
w = F.window(F.col("event_time"), "1 minute")
agg = (scored
       .withWatermark("event_time", "2 minutes")
       .groupBy(w.alias("window"), F.col("stream_id"))
       .agg(F.count("*").alias("n_comments"),
            F.sum((F.col("pred_label") == "negative").cast("int")).alias("n_negative"),
            F.sum((F.col("pred_label") == "neutral").cast("int")).alias("n_neutral"),
            F.sum((F.col("pred_label") == "positive").cast("int")).alias("n_positive"),
            F.round(F.avg("confidence"), 4).alias("avg_confidence"),
            F.round(F.avg("latency_ms"), 1).alias("avg_latency_ms")))
```

8.4. DLQ — hàng đợi lỗi

Mọi bản ghi không hợp lệ được gán một `dlq_reason` và chuyển sang DLQ, không bị bỏ âm thầm. Các lý do được kiểm tra theo thứ tự: JSON hỏng, thiếu trường bắt buộc, `event_ts` không đọc được, văn bản rỗng sau làm sạch, quá ngắn hoặc quá dài. Bản ghi hợp lệ là bản ghi có `dlq_reason IS NULL`.

Bảng 19. Lý do DLQ trong lần chạy verify1

Lý do	Số bản ghi	Ca kiểm chứng
json_khong_hop_le	3	JSON hỏng, JSON mảng, JSON null
thieu_truong:text	1	Thiếu trường text
thieu_truong:comment_id	1	Thiếu comment_id
thieu_truong:event_ts	1	Thiếu event_ts
event_ts_khong_doc_duoc	1	Chuỗi thời gian không hợp lệ
van_ban_rong_sau_lam_sach	1	Toàn khoảng trắng
van_ban_qua_ngan_duoi_20_ky_tu	1	Chuỗi “ok”
van_ban_qua_dai_tren_5000_ky_tu	1	Vượt ngưỡng max_chars
Tổng	10	—

Điểm quan trọng: **10 ca lỗi được cố ý gửi** đều xuất hiện đúng trong DLQ với lý do tương ứng. Ngoài ra, ở một lần chạy thử với 30 bình luận, có một bình luận thật bị đưa vào DLQ vì dài quá 5.000 ký tự — nghĩa là nhánh DLQ bắt được cả dữ liệu thật, không chỉ các ca kiểm thử.

8.5. Exactly-once hay at-least-once?

Đồ án **không tuyên bố exactly-once**. Thực nghiệm mới kiểm chứng:

- Không có bản ghi nào bị mất trong lần chạy `verify1`: 1.014 bản ghi đã chấm + 10 bản ghi DLQ = 1.024 bản ghi đã gửi.
- Dừng job êm hai lần, cả 6 query dừng sạch và checkpoint được ghi.
- Chưa thử **job chết giữa micro-batch** rồi khởi động lại; chưa đo xem sink có bị ghi trùng hay không.

Vì vậy, phát biểu đúng là: hệ thống đã được kiểm chứng **at-least-once** trong các lần chạy thử; **exactly-once chưa được kiểm chứng**. Kafka sink và Parquet sink có cơ chế riêng, nhưng đồ án không đủ bằng chứng để khẳng định chúng chống trùng trong mọi tình huống.

8.6. Quyền riêng tư trong streaming

Job streaming mặc định chạy với `privacy.keep_text: false`: văn bản đã làm sạch không được ghi vào sink, chỉ giữ `comment_id`, `stream_id`, `user_hash`, nhãn dự đoán, confidence và latency. Cờ `--keep-text` chỉ được dùng trong demo với dữ liệu IMDb công khai, và script `run_demo.sh` ghi rõ điều này trong phần mô tả. Khi thu bình luận thật, không được dùng cờ này.

`user_hash` là SHA-256 có muối, cắt còn 16 ký tự hex. Muối đọc từ biến môi trường `COMMENT_HASH_SALT` (lưu ngoài repo), nên không thể đảo ngược bằng cách thử toàn bộ không gian tên người dùng nếu không có muối.

8.7. Vận hành

Các script vận hành được thiết kế để chạy trên máy ảo cá nhân, không cần cluster:

- `scripts/env.sh` — nạp biến môi trường (Java 8 cho Spark, Java 17 cho Kafka, Spark 3.5.9, Kafka 4.3.1, Python 3.11).
- `kafka/start_kafka.sh` — khởi động Kafka KRaft; `kafka/create_topics.sh` tạo ba topic với số partition cần thiết.
- `scripts/run_stream.sh <tên>` — chạy job ở nền, chờ đến khi READY rồi mới trả về; điều này tránh producer gửi trước khi job gắn offset.
- `scripts/stop_stream.sh <tên>` — dừng êm, gửi SIGTERM để job hoàn tất micro-batch hiện tại và ghi checkpoint.
- `scripts/run_demo.sh` — chạy toàn bộ demo bằng một lệnh; `--stop` để dừng.

Một chi tiết vận hành đã gặp thật: nếu HDFS đang ở safe mode, Spark có thể chết khi tạo event log. `run_stream.sh` kiểm tra HDFS và tự tắt event log cho riêng lần chạy nếu cần, thay vì sửa `spark-defaults.conf` dùng chung.

9. Kiểm chứng streaming

9.1. Kết quả đối chiếu

Lần chạy chính thức `verify1` diễn ra ngày 15/10/2026. Producer gửi 1.000 bình luận từ tập test với tốc độ 50 bình luận/giây, cộng thêm 14 bản ghi kiểm chứng hợp lệ (trong đó có một bản ghi cố ý trễ 10 phút và 12 bản ghi nhịp tim) và 10 bản ghi lỗi cố ý. Tổng cộng **1.024 bản ghi**.

Bảng 20. Đối chiếu số lượng và chất lượng (stream_verification.csv)

Hạng mục	Kết quả	Ghi chú
Tổng gửi	1.024	1.000 replay + 14 kiểm chứng + 10 lỗi
Đã chấm điểm (scored_detail)	1.014	1.000 + 14 hợp lệ
Vào DLQ	10	Đúng 10 ca lỗi cố ý
Cân bằng	1.014 + 10 = 1.024	PASS
Kafka comments_scored	+1.014	Chênh lệch offset
Kafka comments_dlq	+10	Chênh lệch offset
Số micro-batch	110	Trên 6 query
Số query	6	detail, kafka, window, sliding, dlq_kafka, dlq_parquet
Dự đoán luồng khớp offline	1.000/1.000	100,00% trên cùng comment_id
Macro-F1 trên 1.000 dòng	0,6311	Offline cùng 1.000 dòng: 0,6311
Macro-F1 offline toàn test	0,6233	Chênh +0,0078
Accuracy trên luồng	0,7030	1.000 dòng
Weighted-F1 trên luồng	0,7230	1.000 dòng

Điểm mạnh nhất của kiểm chứng này là **so sánh trên cùng `comment_id`**: 1.000 bản ghi được chấm cả trên luồng lẫn offline, và cả 1.000 dự đoán đều giống nhau. Điều này chứng minh rằng hàm làm sạch, thứ tự stage và mô hình trong streaming **khớp chính xác** với lúc huấn luyện — không có lệch phân phối do xử lý khác nhau.

Chỉ số trên 1.000 dòng cao hơn toàn bộ test (macro-F1 0,6311 so với 0,6233) không phải vì mô hình tốt hơn, mà vì mẫu 1.000 dòng có phân bố lớp khác: 628 positive, 230 negative, 142 neutral. Đây là lý do phải so sánh trên **cùng một tập id**, không so sánh con số tuyệt đối giữa hai tập khác nhau.

9.2. Chỉ số theo lớp trên luồng

Bảng 21. Precision / Recall / F1 trên 1.000 dòng streaming

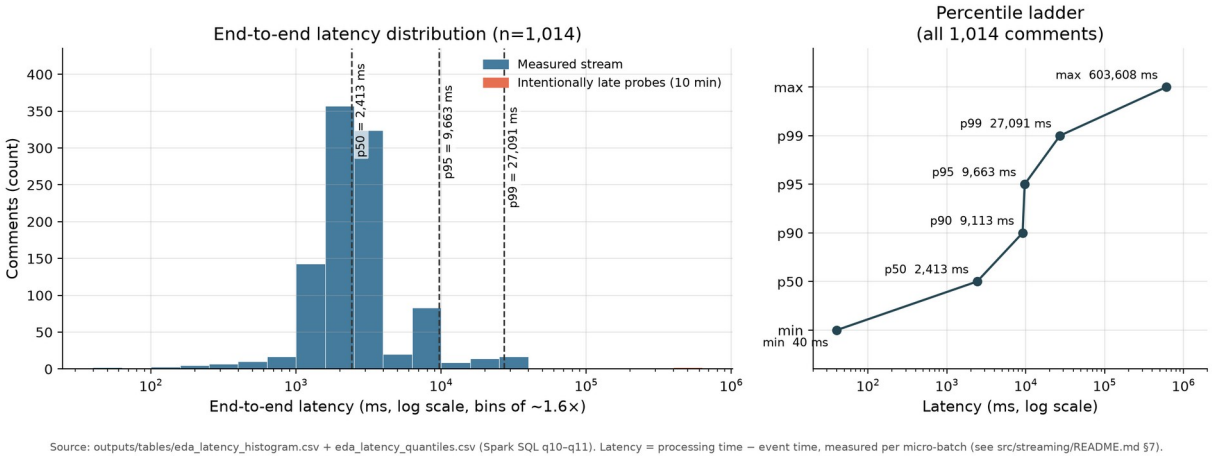
Lớp	Precision	Recall	F1	Support
Negative	0,7064	0,6696	0,6875	230
Neutral	0,3141	0,5352	0,3958	142
Positive	0,8759	0,7532	0,8099	628

So với offline toàn test, F1 negative và neutral trên mẫu 1.000 cao hơn một chút (0,6875 so với 0,6723; 0,3958 so với 0,3783), còn positive thấp hơn một chút (0,8099 so với 0,8193). Đây là dao động lấy mẫu, không phải cải thiện thật. Kết luận đúng: **trên cùng một tập id, luồng và offline cho kết quả giống hệt nhau**.

9.3. Độ trễ

Bảng 22. Phân bố độ trễ end-to-end (ms)

Chỉ số	Giá trị	Diễn giải
Min	40	Bản ghi đến khi job đang chờ
p50	2.413	Một nửa số bình luận được chấm dưới 2,4 giây
p95	9.663	5% bình luận chậm hơn 9,7 giây
p99	27.091	1% bình luận chậm hơn 27 giây
Max	603.608	Hai bản ghi cố ý trễ 10 phút
Kafka p50	0	Chặng Kafka gần như tức thời
Queue p50	2.355	Phần lớn độ trễ nằm ở hàng đợi xử lý
Queue p95	9.136	Hàng đợi là nút thắt chính



Hình 7. Phân bố độ trễ end-to-end; đuôi dài đến từ hàng đợi xử lý, không phải từ Kafka.

Phân bố có **hai đỉnh**: 824/1.014 bình luận (81%) được chấm trong 1,0–4,0 giây, nhưng có 83 bình luận rơi vào 6,3–10,0 giây. Nguyên nhân là bình luận dồn về cùng lúc khi producer tăng tốc; Kafka gần như không đóng góp vào độ trễ (p50 = 0 ms), còn hàng đợi trong micro-batch đóng góp phần lớn (p50 = 2.355 ms, p95 = 9.136 ms).

Độ trễ p95 9,7 giây là **quá chậm cho phản hồi tức thì** trong livestream. Nếu cần hiển thị cảm xúc ngay, phải giảm trigger từ 5–10 giây xuống 1–2 giây hoặc tăng mức song song. Đồ án chưa tối ưu vì mục tiêu là kiểm chứng tính đúng đắn trước, hiệu năng sau.

Hai bản ghi có độ trễ 603 giây là **cố ý**: chúng được gửi với `event_ts` trễ 10 phút để kiểm tra watermark. Chúng không đại diện cho độ trễ bình thường và phải được loại khỏi phân tích hiệu năng. Nếu bỏ hai bản ghi này, đuôi dài vẫn còn do hàng đợi, nhưng max giảm mạnh.

9.4. Watermark và bản ghi đến muộn

Watermark 2 phút có tác dụng: bản ghi `probe-late` được gửi sau khi watermark đã tiến 10 phút **không xuất hiện** trong cửa sổ 1 phút, trong khi bản ghi `bad-11` đến cùng batch đầu thì có mặt. Điều này được kiểm chứng bằng **đối chiếu dữ liệu**, không chỉ bằng log.

> **Một điểm không nhất quán đã ghi nhận**
>
> Spark báo `numRowsDroppedByWatermark = 0` cho cả 6 query, nhưng dữ liệu lại cho thấy `probe-late` bị loại khỏi cửa sổ. Đồ án ghi rõ sự không nhất quán này thay vì che đi. Kết luận về watermark dựa trên **so sánh dữ liệu đầu ra**, không dựa vào chỉ số nội bộ của Spark.

9.5. Những gì đã và chưa kiểm chứng

Bảng 23. Trạng thái kiểm chứng

Điều đã kiểm chứng	Bằng chứng
Không mất bản ghi trong lần chạy verify1	1.014 + 10 = 1.024
DLQ bắt đúng 10 ca lỗi cố ý	stream_dlq_rows.csv
Luồng cho cùng dự đoán với offline	1.000/1.000 khớp
Dừng êm 6 query	2 lần dừng sạch, checkpoint ghi
Watermark loại bản ghi quá muộn	probe-late không có trong cửa sổ

Bảng 24. Những gì CHƯA kiểm chứng

Điều chưa kiểm chứng	Ảnh hưởng
Exactly-once khi job chết giữa micro-batch	Có thể ghi trùng hoặc mất khi restart
Tải cao hơn 384 msg/s	Chưa biết hành vi và độ trễ
Bình luận livestream thật (tiếng Việt, teencode)	Chỉ số có thể giảm mạnh
Chạy nhiều ngày liên tục	Chưa đo memory leak, state growth

10. Phân tích khám phá dữ liệu và dashboard

10.1. Phạm vi EDA

Phân tích khám phá dữ liệu (EDA) được chạy bằng **Spark SQL** — 13 truy vấn trong `src/eda/sql/`, kết quả ghi ra `outputs/tables/eda_*.csv`, rồi vẽ **15 biểu đồ PNG + 1 biểu đồ HTML tương tác** trong `outputs/figures/`. Mọi insight được tổng hợp trong `outputs/insights.md`; các con số dưới đây lấy từ file đó và từ các bảng `eda_*.csv`.

Phạm vi số liệu gồm hai phần: **dữ liệu huấn luyện** (129.701 train / 27.793 validation / 27.792 test) và **đầu ra streaming** của lần chạy `verify1` (1.014 bình luận). Phần streaming là **replay**, không phải dữ liệu livestream thật.

10.2. Các insight chính

Bảng 25. Insight EDA và hành động tương ứng

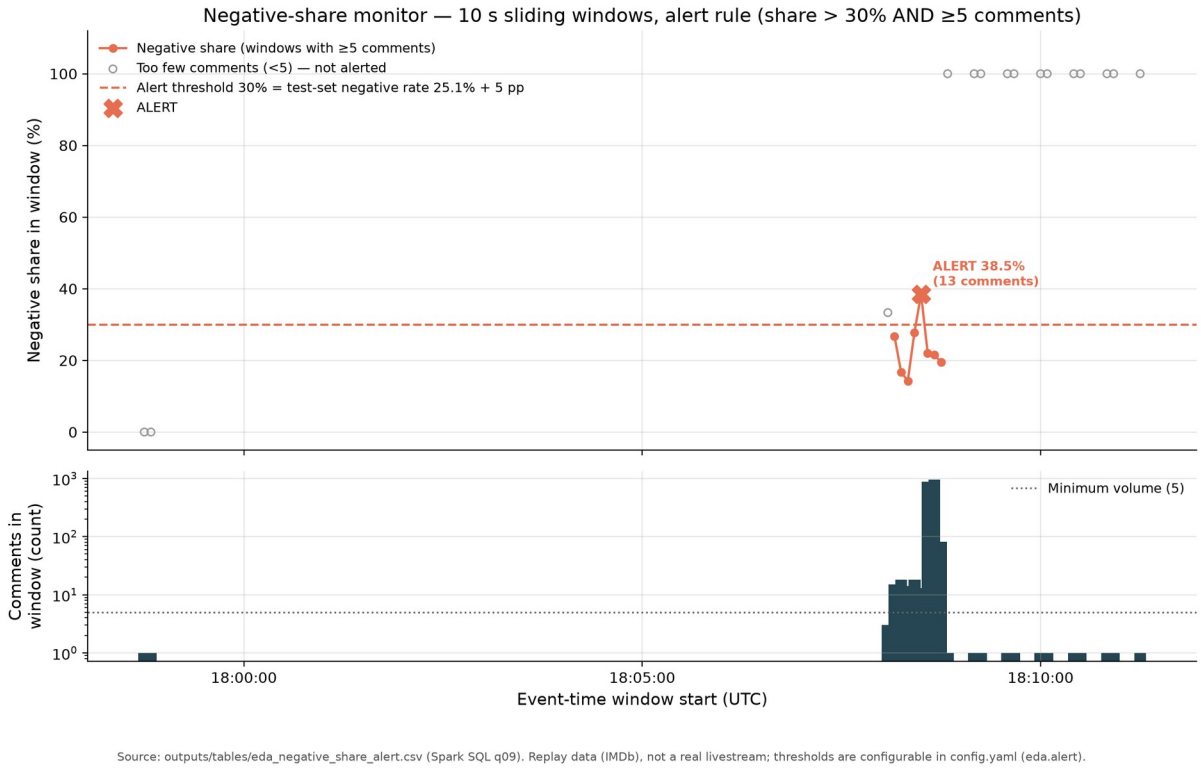
Chủ đề	Con số thật	Hành động
Mất cân bằng lớp	Positive 61,1% (79.214 train); negative 25,1%; neutral 13,9%	Không dùng accuracy; theo dõi F1 từng lớp
Độ dài bình luận	Neutral trung vị 131 token, p95 501; positive 83; negative 73	Ưu tiên đọc bình luận dài khi kiểm duyệt
Từ đặc trưng	Negative “bad” z=80,4; neutral “6/10” z=25,3; positive “great” z=63,1	Cảnh giác mô hình học điểm số
Emoji	Positive 46/1.000 vs neutral 18/1.000; 2,1% positive có emoji	Không dùng emoji làm quy tắc cảm xúc
Thời gian	969/1.014 bình luận đến trong 27 giây (18:08:18–18:08:45 UTC)	Dùng cửa sổ trượt 10 giây để bắt diễn biến
Thông lượng	Đỉnh 384 msg/s = 96% trần thiết kế 400 msg/s	Cảnh báo hạ tầng ở 400 msg/s
Cảnh báo	1 cảnh báo đúng lúc 18:08:30 UTC: 38,5%, 13 bình luận	Giữ điều kiện tối thiểu 5 bình luận
Độ trễ	81% trong 1–4 giây; p50 2.413 ms; p95 9.663 ms	Hạ trigger nếu cần phản hồi tức thì
Chất lượng mô hình	F1 negative 0,67; neutral 0,38; positive 0,82	Không dùng ngưỡng chung cho ba lớp

10.3. Dashboard Streamlit

Dashboard nằm ở `src/app/app.py`, chạy bằng Streamlit, đọc Parquet do job streaming ghi ra. Dashboard **không cần Spark** và **không ghi** vào bất kỳ nguồn dữ liệu nào — nó chỉ đọc. Giao diện có ba tab:

- **Live Monitor:** số bình luận theo lớp, tỷ lệ tiêu cực theo cửa sổ, danh sách bình luận mới nhất (nếu bật `--keep-text`), cảnh báo khi vượt ngưỡng.
- **Try It:** nhập một câu bất kỳ, chạy qua PipelineModel và xem nhãn + xác suất; có bốn câu ví dụ lấy từ `config.yaml`.
- **About:** giải thích kiến trúc, dữ liệu, giới hạn và tuyên bố trung thực.

Quy tắc cảnh báo: **tỷ lệ tiêu cực > 30% và số bình luận ≥ 5** trong cửa sổ 10 giây. Ngưỡng 5 bình luận là để tránh báo động giả từ các cửa sổ chỉ có 1–3 bản ghi nhịp tim. Trong lần chạy `verify1`, quy tắc bắn đúng **1 lần** lúc 18:08:30 UTC với 38,5% và 13 bình luận — đúng đỉnh tiêu cực thật của đợt bùng nổ. Nếu bỏ điều kiện 5 bình luận, sẽ có thêm **13 báo động giả** với tỷ lệ 33–100%.



Hình 8. Tỷ lệ tiêu cực theo cửa sổ 10 giây và cảnh báo duy nhất của lần chạy verify1.

Dashboard là công cụ **trình diễn và giải thích**, không phải hệ thống giám sát sản xuất. Nó không có phân quyền, không có cảnh báo qua email/SMS, không có lưu lịch sử cấu hình. Những điều này nằm ngoài phạm vi đồ án.

10.4. Những điều EDA KHÔNG chứng minh

EDA trên dữ liệu replay chỉ minh họa **cơ chế** của hệ thống, không phải hành vi của người xem thật. Cụ thể:

- Tỷ lệ tiêu cực 38,5% là do cách chọn mẫu replay, không phải một đợt bùng phát thật ngoài đời.
- Đỉnh 384 msg/s là do cấu hình `burst_factor`, không phải tải thật của một livestream.
- Phân bố emoji và từ đặc trưng phản ánh review IMDb, không phản ánh bình luận livestream.
- Không có kết luận nào về tiếng Việt, teencode hay văn hóa bình luận Việt Nam — bộ dữ liệu hiện tại là tiếng Anh.

11. Chương trình thử nghiệm mô hình

11.1. Mục đích

`src/app/model_trial.py` là một chương trình nhỏ để **thử nghiệm mô hình mà không cần Kafka hay Streamlit**. Nó nạp PipelineModel đã lưu, chấm một câu tự nhập, một danh sách câu, hoặc một mẫu từ tập test, rồi in kết quả và ghi bằng chứng ra CSV. Chương trình này phục vụ báo cáo và demo, không phải một ứng dụng hoàn chỉnh.

Bảng 26. Các chế độ của chương trình thử nghiệm

Chế độ	Lệnh	Kết quả
Tương tác	`bash scripts/run_model_trial.sh`	Nhập từng câu, xem nhãn + xác suất
Ví dụ	`bash scripts/run_model_trial.sh --examples`	Chấm 4 câu ví dụ trong config
Một câu	`bash scripts/run_model_trial.sh --text "not bad"`	Chấm một câu
Mẫu test	`bash scripts/run_model_trial.sh --test-sample 200`	Đo accuracy/F1 trên mẫu

Chương trình chạy bằng Python của venv (`$VENV_PY`) thay vì gọi trực tiếp `spark-submit`, vì `spark-submit` ở chế độ client nuốt stdin khiến chế độ tương tác không nhận được câu người dùng gõ. Chương trình tắt event log cho lần chạy để không phụ thuộc HDFS.

11.2. Kết quả mẫu 200 dòng

Lần chạy `--test-sample 200` trên tập test cho kết quả:

Bảng 27. Kết quả chương trình thử nghiệm trên 200 dòng test

Chỉ số	Giá trị
Accuracy	0,7200
Macro-F1	0,6317
Weighted-F1	0,7380
F1 negative	0,6847
F1 neutral	0,3750
F1 positive	0,8356
Số dòng	200
Dòng lỗi	0

Các chỉ số này **không thay thế** kết quả trên toàn bộ 27.792 dòng test. Mẫu 200 dòng có sai số lấy mẫu lớn; accuracy 0,72 so với 0,7018 toàn tập là khác biệt bình thường. Khi báo cáo, phải nói rõ đây là **mẫu 200 dòng**, không được trình bày như kết quả cuối cùng.

Chế độ `--examples` chấm bốn câu ví dụ trong `config.yaml`: “not bad at all actually, quite enjoyable”; “it was okay, nothing special. 6/10”; “the acting is good. the story is very much lacking”; “his 'bad acting' had me crying with laughter, absolutely brilliant”. Đây là các câu được chọn để minh họa phủ định, điểm số, khen chê lẫn lộn và mỉa mai — không phải mẫu đại diện thống kê.

11.3. Cách dùng trong buổi báo cáo

1. Mở terminal, chạy `source scripts/env.sh`.
2. Chạy `bash scripts/run_model_trial.sh --examples` để demo nhanh.
3. Chạy `bash scripts/run_model_trial.sh --text "..."` với một câu do hội đồng đưa ra.

4. Nếu có thời gian, chạy `--test-sample 200` để cho thấy quy trình đo trên dữ liệu thật.

> **Điều cần nói khi demo**
> "Đây là chương trình thử nghiệm mô hình, không phải app hoàn chỉnh. Nó không cần Kafka, không cần dashboard; mục đích là chứng minh mô hình đã lưu có thể nạp lại và cho dự đoán nhất quán. Kết quả trên 200 dòng chỉ là minh họa; kết quả chính thức nằm trong báo cáo với 27.792 dòng test."

12. Hướng dẫn chạy lại

12.1. Yêu cầu môi trường

Bảng 28. Thành phần môi trường

Thành phần	Phiên bản / đường dẫn	Ghi chú
Java cho Spark	Java 8	`JAVA_HOME=/usr/lib/jvm/java-8-openjdk-arm64`
Spark	3.5.9	`SPARK_HOME=/home/hadoop/spark-3.5.9`
Kafka	4.3.1 (KRaft)	`KAFKA_HOME=/home/hadoop/kafka`, Java 17
Python cho Spark	3.11	`PYSPARK_PYTHON=/usr/bin/python3.11`
Python cho venv	3.14	`.venv/bin/python`
HDFS	Tuỳ chọn	Chỉ cần cho event log và một số test
Biến môi trường	`COMMENT_HASH_SALT`	Đọc từ `~/.capstone-secrets.sh`

Tất cả biến môi trường được nạp bằng `source scripts/env.sh`. Không đặt key, token hay mật khẩu trong code; muối băm đọc từ biến môi trường.

12.2. Các lệnh chính

```
# 0. Nạp môi trường
source scripts/env.sh

# 1. Kiểm tra môi trường
bash scripts/check_env.sh

# 2. Chạy test tự động (cần HDFS đang bật và đã ra safe mode)
.venv/bin/python -m pytest tests/ -q

# 3. Chuẩn bị dữ liệu (đọc JSON gốc -> parquet)
$VENV_PY src/training/prepare.py

# 4. Huấn luyện và so sánh mô hình
$VENV_PY src/training/train.py --models lr nb svc rf

# 5. Chạy toàn bộ demo (Kafka -> streaming -> producer -> dashboard)
bash scripts/run_demo.sh --duration 600

# 6. Dừng sạch
bash scripts/run_demo.sh --stop

# 7. Thử nghiệm mô hình (không cần Kafka)
bash scripts/run_model_trial.sh --examples
```

12.3. Chạy streaming thủ công

```
# Khởi động Kafka và tạo topic
bash kafka/start_kafka.sh
bash kafka/create_topics.sh
```

```
# Chạy job streaming ở nền, chờ READY
bash scripts/run_stream.sh verify1 --trigger-seconds 5

# Phát 1.000 bình luận từ tập test
$VENV_PY src/producer/replay_producer.py --only-test --max 1000 --rate 50

# Gửi các ca kiểm chứng (heartbeat, late, lỗi cố ý)
$VENV_PY scripts/send_bad_events.py --heartbeat 12 --heartbeat-interval 25 --late-probe

# Dừng êm
bash scripts/stop_stream.sh verify1
```

12.4. Những lỗi thường gặp và cách xử lý

Bảng 29. Lỗi thường gặp

Hiện tượng	Nguyên nhân	Cách xử lý
`ModuleNotFoundError: pyspark`	Chạy bằng Python hệ thống thay vì venv	Dùng `$VENV_PY` hoặc `source scripts/env.sh`
Spark chết khi khởi động	HDFS tắt hoặc đang safe mode	`start-dfs.sh`, chờ safe mode off; hoặc dùng `run_stream.sh` tự tắt event log
Producer gửi nhưng không có kết quả	Job chưa READY, `startingOffsets=latest`	Chờ script `run_stream.sh` báo READY rồi mới gửi
Không thấy bảng trong dashboard	Sai `--run` hoặc sink dir	Kiểm tra `outputs/stream/<run>/` và tham số `--run`
Test lỗi 26 trường hợp	HDFS chưa bật	Bật HDFS, chờ safe mode off
PDF/DOCX lỗi font	Thiếu font trên máy	Cài font hoặc dùng LibreOffice

12.5. Lưu ý về dữ liệu và cấu hình

`config.yaml` hiện tại đặt `prepare.subsample_pct: 64`, nhưng dữ liệu đang dùng là tập 20% tạo bởi code cũ (xem chương 4). Vì vậy:

- Muốn tái lập **đúng** số liệu báo cáo: dùng `data/processed/*.parquet` và `outputs/models/sentiment_pipeline` hiện có; **không** chạy lại `prepare.py` với config 64%.
- Muốn huấn luyện lại trên tập lớn hơn: đổi `subsample_pct` cho phù hợp, chấp nhận rằng tập dữ liệu sẽ khác và mọi chỉ số phải đo lại.
- Không trộn dữ liệu cũ và mới trong cùng một lần huấn luyện — kiểm tra `_manifest.json` trước khi chạy.
- Luôn chạy `--sample N` trước khi chạy bản đầy đủ nếu script hỗ trợ.

13. Hạn chế, rủi ro và hướng phát triển

13.1. Bảng hạn chế

Bảng 30. Hạn chế đã biết, bằng chứng và hướng xử lý

Hạn chế	Bằng chứng	Ảnh hưởng	Hướng xử lý
Nhãn proxy từ rating IMDb	Neutral F1 0,378; 10/30 lỗi do nhãn nhiễu	Trần hiệu năng thấp, đặc biệt neutral	Gán nhãn tay một mẫu; định nghĩa neutral rõ hơn
Khác miền dữ liệu	Review IMDb dài, có điểm số; livestream ngắn, teencode	Chỉ số có thể giảm khi gặp dữ liệu thật	Thu 200–300 bình luận thật, đo lại
Mô hình túi từ	14/30 lỗi do phủ định; mỉa mai	Hiểu sai câu phủ định, mỉa mai	Bigram phủ định, mô hình ngữ cảnh (PhoBERT/BERT)
Học điểm số	“6/10”, “5/10” nằm trong top feature neutral	F1 neutral lạc quan trên IMDb	Loại token điểm số; đo lại trên dữ liệu không có điểm
Chồng lấn người dùng	11.427/111.087 người dùng ở >1 tập; 75.939 dòng (41,0%)	Có thể làm chỉ số lạc quan nhẹ	Chia theo người dùng; báo cáo kèm chỉ số không chồng lấn
Dữ liệu streaming là replay	100% bình luận đến từ tập test IMDb	Không phản ánh hành vi livestream thật	Ghi rõ trong mọi báo cáo; thu dữ liệu thật có phép
Exactly-once chưa kiểm chứng	Chưa thử job chết giữa micro-batch	Có thể ghi trùng khi restart	Thử crash test; đo sink; ghi rõ at-least-once
Độ trễ p95 9,7 giây	Queue p50 2.355 ms, p95 9.136 ms	Chưa đủ nhanh cho phản hồi tức thì	Giảm trigger, tăng song song, tối ưu state
Chỉ chạy local[2] trên một máy	Cấu hình Spark local	Không đại diện cho cluster	Đo trên cluster; tách driver/executor
Chưa có ảnh nhóm	Repo không có ảnh đội	Báo cáo tiếng Anh để chỗ trống	Chụp ảnh nhóm và chèn trước khi nộp
Tiến độ chậm so với kế hoạch	Day 20 đã qua, một số hạng mục còn mở	Cần báo cáo trung thực	Nêu rõ trong PROGRESS và báo cáo

13.2. Rủi ro khi trình bày

- **Đừng nói “mô hình đạt 70%” mà không nói baseline 61%.** Người nghe có thể hiểu sai mức cải thiện. Phải nói kèm macro-F1 và F1 từng lớp.
- **Đừng gọi dữ liệu replay là livestream thật.** Nói rõ: “Đây là review IMDb phát lại để kiểm chứng cơ chế; chưa phải dữ liệu livestream.”
- **Đừng khẳng định exactly-once.** Nói: “Đã kiểm chứng at-least-once; exactly-once chưa kiểm chứng.”
- **Đừng nói mô hình “hiểu” cảm xúc.** Nó học tương quan từ túi từ; không hiểu ngữ cảnh, phủ định hay mỉa mai.
- **Đừng giấu lỗi.** 30 mẫu lỗi và F1 neutral 0,378 là bằng chứng trung thực; trình bày chúng sẽ đáng tin hơn là chỉ khoe điểm cao.

13.3. Hướng phát triển

1. **Dữ liệu thật:** thu 200–300 bình luận livestream tiếng Việt có gán nhãn tay, đo lại toàn bộ chỉ số; xin phép nền tảng và tuân thủ hạn mức API.
2. **Mô hình ngữ cảnh:** thử PhoBERT (tiếng Việt) hoặc BERT (tiếng Anh), so sánh với TF-IDF+LR; chấp nhận chi phí huấn luyện cao hơn.
3. **Xử lý phủ định và mỉa mai:** bigram có chủ đích, negation scope, hoặc fine-tune mô hình ngữ cảnh trên các câu phủ định.
4. **Hiệu năng:** giảm trigger xuống 1–2 giây, tăng song song, đo lại p95; thử chạy trên cluster.
5. **Độ tin cậy:** crash test giữa micro-batch, kiểm chứng exactly-once, thêm DLQ replay và giám sát.
6. **Chống chồng lấn:** chia tập theo người dùng, báo cáo thêm chỉ số trên nhóm không chồng lấn.

7. **Sản phẩm:** thêm phân quyền, cảnh báo email/SMS, lưu lịch sử cảnh báo, biểu đồ dài hạn; nhưng phải giữ nguyên tắc không tự động chặn.

Trong ngắn hạn, ưu tiên số một là **đo lại trên dữ liệu thật**. Mọi cải thiện mô hình trên IMDb đều có nguy cơ không chuyển sang livestream; chỉ dữ liệu thật mới trả lời được câu hỏi hệ thống có hữu ích hay không.

14. Phụ lục

14.1. Bản đồ file quan trọng

Bảng 31. Bản đồ file và vai trò

Đường dẫn	Vai trò
`config.yaml`	Toàn bộ tham số: dữ liệu, mô hình, streaming, dashboard
`src/common/text_cleaning.py`	Hàm làm sạch dùng chung cho train và streaming
`src/common/schema.py`	Schema và ánh xạ nhãn dùng chung
`src/training/prepare.py`	Làm sạch, gán nhãn, loại trùng, chia tập
`src/training/train.py`	Pipeline TF-IDF, huấn luyện, đánh giá, lưu model
`src/producer/replay_producer.py`	Phát lại bình luận vào Kafka
`src/streaming/stream_job.py`	6 query streaming, watermark, cửa sổ, DLQ
`src/app/app.py`	Dashboard Streamlit 3 tab
`src/app/model_trial.py`	Chương trình thử nghiệm mô hình
`src/eda/queries.py`, `src/eda/plots.py`	13 truy vấn SQL và 15 biểu đồ
`scripts/env.sh`	Nạp biến môi trường
`scripts/run_demo.sh`	Chạy toàn bộ demo bằng một lệnh
`scripts/run_model_trial.sh`	Chạy thử nghiệm mô hình
`data/processed/`	Dữ liệu đã làm sạch, chia tập (parquet)
`outputs/models/sentiment_pipeline`	PipelineModel đã lưu
`outputs/tables/`	Bảng kết quả, bằng chứng
`outputs/figures/`	Biểu đồ PNG/HTML
`deliverables/Final_Report.docx`	Báo cáo cuối kỳ tiếng Anh (bản nộp)
`docs/bao_cao_ky_thuat_vi.docx`	Tài liệu này

14.2. Thuật ngữ

Bảng 32. Thuật ngữ và giải thích

Thuật ngữ	Giải thích
Event time	Thời điểm sự kiện xảy ra (event_ts), khác processing time
Watermark	Ngưỡng thời gian để đóng cửa sổ cũ và giải phóng state
DLQ	Dead-letter queue — nơi lưu bản ghi lỗi kèm lý do
At-least-once	Bảo đảm mỗi bản ghi được xử lý ít nhất một lần
Exactly-once	Bảo đảm mỗi bản ghi được xử lý đúng một lần
TF-IDF	Term Frequency–Inverse Document Frequency
Macro-F1	Trung bình F1 của các lớp, đối xử bình đẳng
Weighted-F1	Trung bình F1 có trọng số theo số mẫu
Proxy label	Nhãn suy ra gián tiếp từ tín hiệu khác (rating)

Thuật ngữ	Giải thích
Replay	Phát lại dữ liệu lịch sử, không phải luồng trực tiếp
Checkpoint	Nơi lưu offset/state để job khôi phục
Micro-batch	Một lần xử lý nhỏ trong Structured Streaming

14.3. Checklist trước khi báo cáo

- Đã đọc chương 13 (hạn chế) và chuẩn bị câu trả lời cho các rủi ro.
- Đã mở `deliverables/Final_Report.docx` và kiểm tra không có trang trắng.
- Đã chạy `bash scripts/run_model_trial.sh --examples` ít nhất một lần.
- Đã kiểm tra `outputs/tables/` có đủ file trước khi trích số.
- Đã nói rõ dữ liệu streaming là replay, không phải livestream thật.
- Đã nói rõ exactly-once chưa kiểm chứng.
- Đã có ảnh nhóm để chèn vào báo cáo tiếng Anh (nếu nộp bản in).
- Đã kiểm tra GitHub đã đẩy code mới nhất (xem `docs/huong_dan_lay_file_word.md`).

14.4. Tài liệu tham chiếu nội bộ

- `REQUIREMENTS.md` — sản phẩm nộp, mốc Day 1–20, rubric.
- `PROGRESS.md` — nhật ký tiến độ và các hạng mục còn mở.
- `docs/data_dictionary.md` — từ điển dữ liệu chi tiết.
- `docs/bao_cao_tien_do.md` — kịch bản báo cáo tiến độ và 30 câu Q&A.
- `docs/huong_dan_bao_cao.md` — hướng dẫn báo cáo trước hội đồng.
- `outputs/insights.md` — toàn bộ insight EDA kèm số liệu.
- `outputs/streaming_summary.md` — kiểm chứng streaming chi tiết.
- `deliverables/Final_Report.docx` — báo cáo cuối kỳ tiếng Anh.

— Hết tài liệu —